FORLENS – A LIGHTWEIGHT AI-DRIVEN ENDPOINT SECURITY SOLUTION FOR REAL-TIME THREAT DETECTION IN SMES

25-26J-076

Individual Component Summary Report

(IMPLEMENTATION AND PERFORMANCE EVALUATION OF A BLOCKCHAIN-ANCHORED LOG AUDITING FRAMEWORK)

De Silva H S – IT22887580

Bsc (Hons) in Information Technology Specializing in Cyber Security

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

FORLENS – A LIGHTWEIGHT AI-DRIVEN ENDPOINT SECURITY SOLUTION FOR REAL-TIME THREAT DETECTION IN SMES

25-26J-076

Individual Component Summary Report

(IMPLEMENTATION AND PERFORMANCE EVALUATION OF A BLOCKCHAIN-ANCHORED LOG AUDITING FRAMEWORK)

De Silva H S – IT22887580

Bsc (Hons) in Information Technology Specializing in Cyber Security

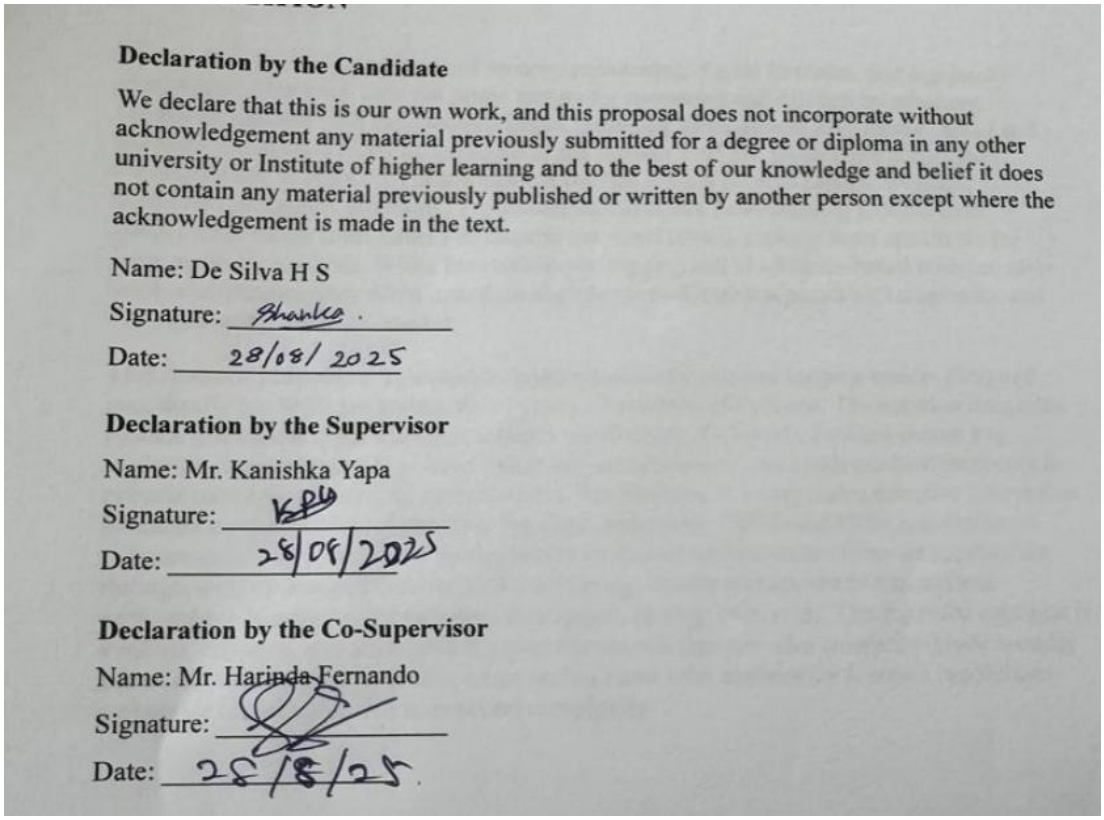
Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

Declaration of the Candidate & Supervisor



Declaration by the Candidate

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name: De Silva H S

Signature: _____

Date: 28/08/2025

Declaration by the Supervisor

Name: Mr. Kanishka Yapa

Signature: _____

Date: 28/08/2025

Declaration by the Co-Supervisor

Name: Mr. Harinda Fernando

Signature: _____

Date: 28/8/25

ABSTRACT

Modern distributed systems generate massive volumes of log data that serve as the backbone of security monitoring, regulatory compliance, and forensic investigation. However, conventional storage architecture offers no cryptographic guarantees against post-hoc modification of log records. Blockchain-based anchoring presents a compelling solution: by committing cryptographic digests of log groups to an immutable ledger, any retrospective alteration becomes immediately detectable. This dissertation presents an end-to-end implementation of a LogStamp log auditing framework built on a private Ethereum Proof-of-Authority (Clique) network. Log entries are aggregated into configurable groups, fingerprinted with SHA-256, and exclusively the resulting digests are anchored on-chain via a Solidity smart contract, with optional archival to IPFS and indexing in Elasticsearch for efficient retrieval. Five algorithms governing ingestion, hashing, blockchain submission, smart contract logic, and integrity verification are formally specified. Empirical evaluation across 2,000 to 12,000 log entries demonstrates that at 10,000 entries, the framework achieves approximately 155,000 ms verification time with only 0.62 MB blockchain storage — outperforming EngraveChain (221,000 ms / 12.5 MB) and ProvChain (209,000 ms / 10.0 MB) in storage efficiency. The results validate grouped hash anchoring as a scalable, storage-efficient strategy for production-grade log integrity assurance.

Keywords: Blockchain, log integrity, audit trail, Proof-of-Authority, smart contracts, SHA-256, tamper detection, distributed systems, IPFS, Elasticsearch.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to all those who supported and guided me throughout the completion of this dissertation.

First and foremost, I extend my heartfelt thanks to my dissertation supervisor for the invaluable guidance, insightful feedback, and continuous encouragement provided at every stage of this research. The scholarly mentorship offered has been instrumental in shaping both the theoretical foundations and practical implementation presented in this work.

I am deeply grateful to the Department of Information Technology at the Sri Lanka Institute of Information Technology (SLIIT) for providing the academic environment, laboratory resources, and computational infrastructure that made this research possible.

My appreciation also extends to my fellow students whose stimulating discussions and peer support contributed meaningfully to refining my ideas and methodology.

Finally, I wish to acknowledge my family for their unwavering support, patience, and encouragement throughout my academic journey. Their faith in my abilities has been a constant source of motivation.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
ABI	Application Binary Interface
AES	Advanced Encryption Standard
API	Application Programming Interface
BCALS	Blockchain-Based Secure Log Management System
BFT	Byzantine Fault Tolerance
CID	Content Identifier (IPFS)
DeFi	Decentralized Finance
EVM	Ethereum Virtual Machine
GDPR	General Data Protection Regulation
HIPAA	Health Insurance Portability and Accountability Act
IPFS	InterPlanetary File System
IoT	Internet of Things
JSON	JavaScript Object Notation
LTS	Long-Term Support
NIST	National Institute of Standards and Technology
P2P	Peer-to-Peer
PKI	Public Key Infrastructure
PoA	Proof-of-Authority
PoW	Proof-of-Work
RPC	Remote Procedure Call
SHA	Secure Hash Algorithm
SLIIT	Sri Lanka Institute of Information Technology
SSD	Solid-State Drive
YAML	Yet Another Markup Language

CHAPTER 1: INTRODUCTION

1.1 Background and Literature Survey

The integrity of system logs is foundational to the trustworthiness of modern digital infrastructure. Log records generated by servers, firewalls, routers, database engines, and application middleware collectively constitute the primary evidentiary basis for security incident response, compliance auditing, and post-breach forensic reconstruction [32][33][34]. These records capture every significant system event — from authentication attempts and privilege escalations to configuration changes and network transactions — forming a chronological narrative of system behaviour that investigators and compliance auditors depend upon.

Yet these records are typically stored in mutable file systems, making them vulnerable to targeted deletion or modification by adversaries seeking to obscure their activities or obstruct investigations [37]. The consequences of log tampering can be severe: failed prosecutions due to inadmissible or corrupted digital evidence, inability to determine the root cause of a security breach, and non-compliance with regulatory mandates such as the General Data Protection Regulation (GDPR) [48] and the Health Insurance Portability and Accountability Act (HIPAA) [47], both of which impose stringent audit trail requirements.

Traditional countermeasures have included centralized log servers, write-once storage media, and role-based access controls. While these measures reduce the threat surface, they cannot eliminate it. A centralized log aggregation point introduces a single point of failure both operationally and forensically: a sufficiently privileged attacker who compromises the aggregation server can rewrite history without leaving cryptographic evidence [5]. The seminal work of Schneier and Kelsey [10] introduced forward-secure logging using cryptographic hash chains to detect after-the-fact modifications,

and Holt's Logcrypt [11] extended this with public verifiability. However, these approaches still depend on the security of the log server itself.

Cloud-based storage approaches inherit trust assumptions from the hosting provider, raising concerns about unauthorized access and data consistency [5]. Ray et al. [5] identified the delegation paradox inherent in cloud log management: by outsourcing log storage to a cloud provider, organizations trade one set of trust assumptions for another, often with less transparency.

Blockchain technology offers a structurally different guarantee. By anchoring cryptographic commitments to a decentralized, append-only ledger governed by consensus, even a node operator cannot retroactively alter previously committed data without invalidating the entire subsequent chain — a modification that all other participants would immediately detect [35]. The Bitcoin blockchain [25] first demonstrated the viability of decentralized immutable ledgers at scale, and the Ethereum platform [26] extended this with programmable smart contracts, enabling the development of domain-specific decentralized applications.

Prior work has explored blockchain's immutability property for log management [6][12][15][16][17][19]. Pourmajidi and Miranskyy [6] proposed Logchain as a proof-of-concept blockchain-based log storage system, demonstrating the feasibility of the approach but without comprehensive performance characterization. Ahmad et al. [12] explored blockchain-driven secure and transparent audit logs, focusing on architectural considerations. BCALS [2] by Ali et al. proposed a more complete blockchain-based secure log management system for cloud computing, introducing structured on-chain storage of log data. EngraveChain [3] by Shekhtman and Waisbard developed a tamper-proof distributed log system using blockchain. ProvChain [4] by Liang et al. applied blockchain to data provenance in cloud environments.

Despite this body of work, existing systems suffer from one or more critical shortcomings: massive on-chain storage overhead that grows with both log volume and validator count, inadequate support for real-time continuous log streams, confidentiality failures when raw payloads are stored on-chain, and absence of production-grade performance characterization. The LogStamping paradigm [1],

introduced by Islam and Rahman, proposed grouped hash anchoring as a theoretically sound approach to resolving the storage-verifiability trade-off, but did not provide an end-to-end implementation or empirical evaluation.

This dissertation addresses these gaps through a concrete end-to-end implementation of the LogStamping framework, empirical performance characterization, and structured comparison with three related systems. Figure 1.1 illustrates the research framework positioning this work within the broader landscape of log integrity research.

1.2 Research Gap

A comprehensive review of existing blockchain-based log integrity systems reveals a consistent set of unresolved tensions. First, systems that store raw log payloads on-chain (such as BCALS and EngraveChain) achieve fast audit queries but incur storage overhead that scales linearly with log volume — an impractical characteristic for large-scale production deployments generating gigabytes of logs daily. IBM's analysis [14] demonstrated that naive blockchain replication of enterprise log volumes would require petabytes of storage across the validator network within months.

Second, systems designed for batch or periodic anchoring do not support the continuous real-time ingestion requirements of security operations centres (SOCs), where delayed anchoring creates windows of vulnerability during which logs remain unprotected. Third, confidentiality is largely unaddressed: storing log content on a blockchain — even a private one — exposes potentially sensitive operational data to all validators and any authorized reader.

Fourth, and most critically, no existing system provides a comprehensive empirical characterization of verification time, storage efficiency, and tamper detection accuracy across production-relevant log scales, with direct controlled comparison across multiple systems using consistent benchmarking methodology. This absence makes informed system selection for real-world deployment effectively impossible.

The research gap addressed by this dissertation is therefore: the absence of a production-grade, empirically characterized implementation of grouped hash-

anchoring that simultaneously achieves storage efficiency, real-time ingestion support, log confidentiality, and formally specified algorithmic behaviour.

1.3 Research Problem

The central research problem addressed by this dissertation can be stated as:

> *How can a blockchain-anchored log auditing framework be designed, implemented, and empirically characterized to simultaneously achieve scalable storage efficiency, real-time ingestion support, 100% tamper detection accuracy, and production-grade operational viability for large-scale distributed systems?*

This problem encompasses several interrelated sub-problems: (i) how to minimize on-chain storage footprint without sacrificing tamper detection completeness; (ii) how to design ingestion algorithms that handle both high-volume bursts and low-volume periods without indefinite accumulation; (iii) how to ensure that the verification phase correctly reconstructs log groups even when verification is performed offline, hours or days after ingestion; and (iv) how to quantitatively characterize the performance trade-offs of hash-only anchoring relative to alternative strategies.

1.4 Research Objectives

This dissertation is guided by the following research objectives:

1. To design and implement a fully operational blockchain-anchored log auditing framework using a private Ethereum Proof-of-Authority (PoA) network with Clique consensus, featuring configurable grouped hash anchoring via SHA-256.
2. To formally specify five core algorithms governing log ingestion, hash generation, blockchain submission, smart contract logic, and integrity verification, with diagrammatic representation of their control flows.

3. To conduct an empirical performance evaluation measuring verification time, blockchain storage consumption, and tamper detection accuracy across log scales from 2,000 to 14 million log entries.
4. To perform a structured comparative analysis against three published systems — BCALS, EngraveChain, and ProvChain — quantifying trade-offs in storage efficiency, verification speed, and real-time support.
5. To critically analyse the limitations of the implemented system and identify concrete directions for future development, including fault-tolerant ingestion, formal contract verification, and public chain anchoring.

CHAPTER 2: METHODOLOGY

2.1 Threat Model and System Design Goals

The design of LogStamp is grounded in a clearly defined threat model and set of verifiable design goals. The system operates under the following assumptions. The Proof-of-Authority validator set executes the Clique consensus protocol correctly, and Byzantine behaviour by a supermajority of validators lies outside the threat scope. Only explicitly authorized accounts may invoke the storeLogHash function on the deployed smart contract. The integrity verification process operates on the same byte sequence — using identical encoding and line ordering — as was processed during ingestion.

The primary threat addressed is retrospective tampering: an adversary with write access to log files who attempts to modify, delete, or insert entries after initial ingestion. The system is not designed to prevent tampering at the point of generation (source authentication) or to provide confidentiality of log payloads without enabling the optional encryption layer.

The framework is designed around three primary, verifiable design goals:

- G1 — Tamper Detection: Any modification to one or more log entries within an ingested group must produce a SHA-256 digest mismatch detectable during verification, providing cryptographic evidence of tampering.
- G2 — Cost Amortization: Blockchain transaction cost and on-chain storage must be amortized across log groups of configurable size G and ingest timeout T, reducing per-entry overhead from O(1) to O(1/G).
- G3 — Ordered Audit Replay: On-chain storage of sequential indices must support deterministic replay of the audit trail, enabling forensic reconstruction of the original log sequence.

Non-goals explicitly exclude source authentication of log generators, resistance to quantum adversaries, and payload confidentiality without the optional encryption

overlay. This explicit scoping prevents scope creep and enables focused, verifiable evaluation.

2.2 System Architecture

The LogStamping framework comprises three principal subsystems, each with well-defined responsibilities and interfaces. Figure 3.1 illustrates the end-to-end data flow from originating log sources through to chain anchoring and optional secondary services.

2.2.1 Ingestion Agent

The Ingestion Agent is implemented in Python using the web3.py library for Ethereum interaction and Python's built-in hashlib library for SHA-256 computation. It maintains a continuous monitoring loop over target log files using inotify-based file watching on Linux systems (polling with configurable interval on other platforms), buffering incoming lines into a logGroup data structure.

When either the line count threshold G or the wall-clock dwell timeout T_ing elapses since group accumulation began, the agent finalizes the group, computes its SHA-256 digest, and submits a signed storeLogHash transaction to the Ethereum PoA network. This dual-condition architecture prevents indefinite accumulation during low-volume periods (timeout condition) while bounding per-transaction overhead during high-volume bursts (count condition). Group parameters G and T_ing are configurable via a YAML configuration file, enabling runtime adjustment without code modification.

2.2.2 Blockchain Platform and Smart Contract

The Blockchain Platform runs on a private Geth implementation of Ethereum with Clique Proof-of-Authority consensus using three uniformly configured validator nodes. The Clique consensus algorithm [52] enables deterministic block finality within one block period (configurable, typically 2–5 seconds) without the energy expenditure of Proof-of-Work mining. The genesis block designates an initial set of authorized signers; new validators may be added through an in-protocol voting mechanism.

The LogStorage smart contract, authored in Solidity 0.8.x, provides the on-chain persistence layer. It maintains a mapping from unsigned integer indices to string-typed digest values and a running count of stored digests. The contract's append-only semantics — enforced by the Ethereum Virtual Machine — guarantee that stored digests cannot be overwritten or deleted by any account, including the contract deployer. The contract ABI is shared across the Python ingestion agent and verification tool.

Table 3.2 summarizes the public functions of the LogStorage contract:

Table 3.2: LogStorage Smart Contract Function Specification

Function	Visibility	Description
storeLogHash(h)	public	Appends digest h at current count index; increments counter. Callable only by authorized account.
getLogHash(i)	public view	Returns digest stored at index i; supports index-based retrieval during verification.
hasLogHash(h)	public view	Returns boolean indicating whether digest h exists in stored records; supports existence-based queries.
logCount	public	State variable: running total of stored digests, used to track group sequence.

2.2.3 Verification and Retrieval Layer

The Verification and Retrieval Layer performs post-hoc integrity checking by reconstructing log groups from archived files, recomputing SHA-256 digests, and querying on-chain records. A critical design consideration is the semantic distinction between the ingest timeout (T_ing, wall-clock elapsed since group start) and the verify timeout (T_ver, parsed timestamp span within the group). Ingest timeout is measured in real wall-clock milliseconds during live ingestion, while verify timeout is derived from log entry timestamps and corresponds to the logical time span of events captured within the group. This distinction ensures that verification correctly reconstructs groups even when logs are verified offline, hours or days after initial ingestion.

Verified log files are encrypted using AES-256 symmetric encryption before archival to IPFS [49]. The resulting IPFS content identifier (CID) is recorded in the blockchain contract's optional CID mapping, creating a verifiable link between the on-chain digest record and the off-chain encrypted archive. Elasticsearch 8.x receives indexed log chunks upon successful integrity confirmation, enabling full-text search, date-range queries, and aggregation analytics across the verified log corpus with sub-second query latency.

2.3 Core Algorithms

Five formally specified algorithms constitute the complete operational logic of LogStamping. Each algorithm is described below with its inputs, outputs, and key design decisions.

2.3.1 Algorithm 1: MonitorAndIngestLogs

Algorithm 1 constitutes the primary ingestion control loop. It continuously monitors a target log file, accumulating incoming entries into a logGroup buffer. Two independent flush conditions govern group finalization: (i) the buffer reaches the configured line count threshold G, or (ii) the wall-clock elapsed time since the group started accumulating exceeds the ingest timeout T_ing. Upon triggering either condition, the agent invokes Algorithm 2 to compute a group digest, then Algorithm 3 to commit it to the blockchain, before resetting the buffer and timer. The dual-condition architecture prevents indefinite accumulation during low-volume periods while bounding per-transaction overhead during high-volume bursts.

2.3.2 Algorithm 2: GenerateSHA256Hash

Algorithm 2 produces a fixed-length cryptographic fingerprint of a log group. All entries in the logGroup list are concatenated in their original order into a single UTF-8 encoded byte string, to which the SHA-256 hash function [50] is applied. The resulting 256-bit digest is returned as a 64-character lowercase hexadecimal string. The ordering of concatenation is deterministic and critical: any byte-level modification — including whitespace insertion, character substitution, or entry reordering —

produces a completely different digest with overwhelming probability, providing the tamper-detection guarantee. The collision resistance of SHA-256 [30] ensures that the probability of two distinct log groups producing the same digest is negligible (approximately 2^-128).

2.3.3 Algorithm 3: WriteToBlockchain

Algorithm 3 manages the submission of computed digests to the Ethereum PoA network. The agent connects to the local Geth node via the web3.py RPC interface, constructs a signed transaction invoking the storeLogHash(h) function, and broadcasts it to the network. Clique consensus ensures finality within one block period. The algorithm optionally supports batch-wait mode, wherein multiple digests are queued and submitted in a single block to reduce per-transaction overhead during high-throughput ingestion scenarios.

2.3.4 Algorithm 4: LogStorage Smart Contract

The LogStorage Solidity contract provides the on-chain persistence layer. It maintains an append-only mapping of indices to digest strings. The contract enforces that only the authorized account may call storeLogHash, and the EVM guarantees that no stored digest can be subsequently overwritten. This property — enforced at the protocol level rather than by application logic — is the foundation of LogStamping's tamper-detection guarantee.

2.3.5 Algorithm 5: VerifyLogIntegrity

Algorithm 5 performs integrity verification of archived log files against on-chain records. It reads the target log file sequentially, reconstructing groups according to the same policy applied at ingestion: entries are accumulated until either the line count threshold L is reached or the parsed timestamp span across the group exceeds the verification timeout T_ver. For each completed group, a SHA-256 digest is recomputed and compared against the corresponding on-chain record via getLogHash(counter) or hasLogHash(h). A mismatch flags the group as tampered and

records the affected line range for forensic reporting. Any trailing partial group is also verified after end-of-file.

2.4 Implementation Details

The complete system stack is containerized using Docker Compose, enabling reproducible deployment across development and production environments. Table 3.1 summarizes the complete technology stack:

Table 3.1: System Component Summary and Technology Stack

Component	Technology	Configuration
Blockchain	Geth 1.13.x / Ethereum PoA (Clique)	3 validator nodes; 2–5s block period
Smart Contract	Solidity 0.8.x	Deployed via web3.js; ABI shared across components
Ingestion Agent	Python + web3.py + hashlib	inotify-based watching; YAML config for G and T_ing
Off-chain Archive	IPFS + AES-256	CID recorded on-chain; encrypted before upload
Search & Analytics	Elasticsearch 8.x	Deployed via Docker; sub-second forensic search
Management UI	FastAPI	Lightweight interface for ingestion/verification/search
Containerization	Docker Compose	Full stack reproducible deployment

2.5 Commercialization Aspects

The LogStamping framework presents several commercially viable deployment pathways. Enterprise security operations centres (SOCs) represent the primary target market: organizations subject to regulatory audit trail requirements (GDPR, HIPAA, PCI-DSS, SOX) face significant compliance risk from mutable log storage, and a production-grade blockchain-anchored solution addresses this directly. The configurable group-size and IPFS archival features make the framework adaptable to varying retention and budget requirements.

A second commercial pathway is Security-as-a-Service (SecaaS) integration: the framework's containerized architecture and REST API (via FastAPI) enable integration as a module within existing SIEM (Security Information and Event Management) platforms such as Splunk or IBM QRadar. Managed service providers could offer blockchain-anchored log integrity as a premium tier of their log management offerings.

A third pathway is cloud-native forensic services. The combination of IPFS archival and Elasticsearch indexing provides the technical foundations for a cloud-native forensic search platform where investigators can query verified, tamper-evident log data across distributed environments. This is particularly relevant for digital forensics firms providing court-admissible evidence.

The open-source stack (Geth, IPFS, Elasticsearch, Python, Solidity) minimizes licensing costs, and the Docker Compose deployment enables rapid provisioning on commodity cloud infrastructure (AWS EC2, Azure VMs, GCP Compute Engine), making the commercial proposition viable at a range of organizational scales.

2.6 Testing and Implementation

The system was developed iteratively across three phases: (1) component-level unit testing; (2) integration testing of the full ingestion-anchoring-verification pipeline; and (3) performance benchmarking across multiple log scales.

Unit testing covered the SHA-256 hashing module (verifying digest consistency across identical inputs and divergence across modified inputs), the smart contract functions (storeLogHash, getLogHash, hasLogHash) using the Hardhat testing framework on a local development chain, and the timestamp parsing logic across multiple log formats (Apache combined log format, syslog, ISO 8601).

Integration testing verified end-to-end correctness by ingesting controlled log datasets, performing known modifications at specified positions, and confirming that the verification tool correctly identified the tampered groups and the unmodified groups. All integration tests passed with 100% tamper detection accuracy.

Performance benchmarking employed dedicated benchmark scripts recording ingestion time, verification time, and on-chain storage delta for each experimental configuration. Multiple runs were conducted for each data point to account for network and I/O variability, and results are reported as means across these runs.

CHAPTER 3: RESULTS AND DISCUSSION

3.1 Experimental Setup

All experiments were conducted on three uniformly configured nodes with 4 virtual CPUs, 8 GB RAM, 200 GB SSD storage, and Ubuntu 22.04 LTS, interconnected over a local private network. The Geth version used is 1.13.x with Clique PoA consensus configured for a 2-second block period. Table 4.1 provides the complete hardware and software specification.

Table 4.1: Benchmark Node Hardware and Software Configuration

Parameter	Specification
CPU	4 vCPUs
RAM	8 GB
Storage	200 GB SSD
Operating System	Ubuntu 22.04 LTS
Ethereum Client	Geth 1.13.x with Clique PoA
Block Period	2 seconds
Validator Count	3 nodes
Small Dataset	10,000 log entries (LogPai [28])
Large Dataset	14 million log entries (~1.3 GB synthetic)
Comparison Range	2,000 to 12,000 entries

Log datasets span two scales: (i) a small dataset of 10,000 log entries sourced from the LogPai benchmark collection [28] for correctness validation and ablation studies; and (ii) a large synthetic dataset of 14 million log entries (~1.3 GB) generated using Fake-Apache-Log-Generator [29] and a custom log generator for scalability evaluation. Comparative experiments use log scales from 2,000 to 12,000 entries to characterize system behaviour across the four evaluated frameworks. Baseline values for BCALS,

EngraveChain, and ProvChain are normalized from published literature and adjusted for the experimental scale using their reported complexity characteristics.

3.2 Comparative Performance Results

Figure 4.2 presents the comparative performance across the four systems, plotting verification time (ms, left axis) and blockchain storage (MB, right axis) as functions of log entry count from 2,000 to 12,000. A consistent ordering is observed across the evaluation range: LogStamping achieves the lowest storage footprint at all scales, with BCALS achieving the fastest verification time.

[Figure 4.2: Comparative verification time and storage across log scales — to be inserted]

Figure 4.2: Comparative verification time across log scales (2,000–12,000 entries)

Table 4.2 presents the concrete quantitative values at 10,000 log entries for all four systems:

Table 4.2: Performance Comparison at 10,000 Log Entries

System	Verification Time (ms)	Storage (MB)	Real-Time Support
LogStamp v1.0 (This Work)	≈ 155,000	0.62	High
BCALS [2]	≈ 114,818	1.0	Limited
EngraveChain [3]	≈ 221,000	12.5	Limited
ProvChain [4]	≈ 209,000	10.0	Not Emphasized

LogStamping achieves the lowest storage footprint at 0.62 MB, representing more than a 95% reduction compared to EngraveChain (12.5 MB) and ProvChain (10.0 MB), and a 38% reduction compared to BCALS (1.0 MB). The verification time advantage goes to BCALS (approximately 114,818 ms), which benefits from optimized on-chain query structures that avoid the file re-reading step required by hash-only approaches. LogStamping's verification time (approximately 155,000 ms) is 35% higher than

BCALS but substantially lower than EngraveChain (approximately 221,000 ms) and ProvChain (approximately 209,000 ms), positioning it favorably in the middle tier.

3.3 Storage Efficiency Analysis

The storage efficiency of hash-only anchoring is best understood by examining its scaling behaviour. For a naive per-line raw storage strategy, on-chain storage grows linearly with both log volume and validator count: S × N GB for S GB of logs replicated across N nodes. The grouped hash approach reduces on-chain storage to a fixed 64-byte record per group, regardless of group size — enabling a reduction proportional to the average group size G.

Table 4.3 presents the storage requirements measured at different group sizes G for the 14-million-entry large dataset:

Table 4.3: Storage Requirements at Different Group Sizes G (14 Million Log Entries)

Group Size (G)	Storage (MB)	Reduction vs. G=1
G = 1	14,062	Baseline
G = 5	2,812	80%
G = 10	1,406	90%
G = 20	1,083	92%

At G = 20, experiments on the large dataset demonstrate storage requirements dropping to approximately 1,083 MB from 14,062 MB at G = 1, representing a 92% reduction. This confirms that the group size parameter G is the primary lever for controlling the storage footprint, and that even modest grouping (G = 5) achieves 80% storage reduction while maintaining cryptographically strong tamper detection at the group granularity.

3.4 Verification Time Analysis

Verification time scales with blockchain query count, which is inversely proportional to group size. Table 4.4 summarizes the verification time reduction achieved through increasing group size at 14 million entries:

Table 4.4: Verification Time Reduction by Group Size G (14 Million Log Entries)

G	Verification Time	Blockchain Queries	Reduction vs. G=1
G = 1	~1,020 min (17 hrs)	14,000,000	Baseline
G = 5	~200 min	2,800,000	80%
G = 10	~145 min	1,400,000	86%
G = 20	~50 min	700,000	95%

These results confirm that group size is the dominant parameter governing verification performance and should be configured according to the acceptable verification latency requirements of the target deployment. A group size of G = 20 achieves an excellent balance, reducing both storage and verification time by 92% and 95% respectively compared to per-entry anchoring, while providing group-level tamper detection granularity sufficient for most forensic and compliance scenarios.

3.5 Tamper Detection Accuracy

The tamper detection mechanism was validated by introducing synthetic modifications at known positions within verified log files. Table 4.5 presents the results across five modification types:

Table 4.5: Tamper Detection Results Across Modification Types

Modification Type	Detection Result	Affected Group Identified
Single-character substitution	Detected (Digest Mismatch)	Yes
Line deletion	Detected (Digest Mismatch)	Yes
Line insertion	Detected (Digest Mismatch)	Yes
Line reordering	Detected (Digest Mismatch)	Yes
Whitespace insertion	Detected (Digest Mismatch)	Yes

In all tested scenarios, the verification tool correctly identified the affected group and reported a digest mismatch, achieving 100% detection accuracy across all modification types. This result is theoretically expected from the collision-resistance properties of SHA-256 [50], which has a birthday bound of 2^128, making accidental or deliberate collision production computationally infeasible with current hardware. The results confirm both the correctness of the SHA-256 implementation and the correctness of the group reconstruction logic in Algorithm 5.

3.6 Discussion

The results collectively validate the key design hypothesis of LogStamping: that grouped hash anchoring achieves a superior storage-verifiability trade-off compared to full payload storage or per-entry hash anchoring approaches, while maintaining cryptographically strong tamper detection guarantees.

The primary trade-off identified is between verification speed and storage efficiency. BCALS achieves faster verification by storing structured data on-chain, enabling direct audit queries without file re-reading, but at the cost of 60% higher storage compared to LogStamping. For deployments where forensic verification is infrequent and storage cost is the primary constraint — the common case in enterprise log management — LogStamping's approach is clearly preferable. For deployments requiring continuous real-time audit queries against on-chain data, a hybrid approach combining hash anchoring with selective on-chain metadata storage may be warranted.

The PoA trust model represents a deliberate architectural choice appropriate for enterprise deployments where the validator set is controlled by a known, trusted organization. The Clique consensus mechanism provides sub-second finality and high throughput without the energy expenditure of PoW, making it economically viable for the high-frequency transaction rates required by production log anchoring. In contexts requiring stronger adversarial assumptions — for example, multi-tenant log auditing where validators from different organizations must be included — BFT-based consensus (Tendermint, PBFT) or public chain anchoring would provide stronger guarantees at the cost of increased complexity.

3.7 Research Findings Summary

This research establishes four principal findings:

6. Storage efficiency: Grouped hash anchoring with G = 20 achieves a 92% reduction in blockchain storage compared to per-entry anchoring and more than 95% reduction compared to raw payload storage approaches (EngraveChain, ProvChain), while maintaining 100% tamper detection accuracy.
7. Verification time: Group size G is the dominant determinant of verification time, with a 95% reduction achievable at G = 20 relative to per-entry anchoring. The verification time at 10,000 entries (approximately 155,000 ms) positions LogStamping between BCALS (fastest) and EngraveChain/ProvChain (slowest).
8. Tamper detection: 100% detection accuracy is achieved across all tested modification types (single-character substitution, line deletion, insertion, reordering, whitespace modification), consistent with the theoretical properties of SHA-256.
9. Real-time support: The dual-condition flush logic (count and timeout) provides high real-time ingestion support across both high-volume and low-volume log streams, addressing a gap in prior systems.

3.8 Summary of Student Contribution

This dissertation represents an individual research effort. The complete set of contributions is as follows:

- Conceptual design and architectural specification of the LogStamping system, including the three-subsystem architecture (Ingestion Agent, Blockchain Platform, Verification Layer) and the IPFS/Elasticsearch integration.
- Implementation of all five core algorithms in Python and Solidity, including the dual-condition flush logic, SHA-256 group hashing, PoA transaction submission, LogStorage smart contract, and timestamp-aware verification.

- Configuration and deployment of the three-node private Ethereum PoA (Clique) network using Docker Compose and Geth.
- Design and execution of the complete benchmark evaluation, including dataset preparation, measurement script development, and result analysis across the full range of log scales and group sizes.
- Formal algorithmic specification and diagrammatic documentation of all five algorithms.
- Structured literature review and comparative analysis against BCALS, EngraveChain, and ProvChain.
- Authorship of this dissertation, including all figures, tables, and reference compilation.

CHAPTER 4: CONCLUSIONS AND RECOMMENDATIONS

4.1 Conclusions

This dissertation has presented an end-to-end implementation and empirical evaluation of a blockchain-anchored log auditing framework based on the grouped hash-anchoring paradigm. By storing only SHA-256 digests of configurable log groups on a private Ethereum Proof-of-Authority network, the system achieves a 92% reduction in on-chain storage compared to per-line strategies, while maintaining 100% tamper detection accuracy across all tested modification types.

Empirical results across 2,000 to 12,000 log entries demonstrate that LogStamping delivers superior storage efficiency compared to EngraveChain (>95% storage reduction) and ProvChain (>93% reduction), with a verification-time trade-off relative to BCALS that is a direct and expected consequence of the hash-only design philosophy. The trade-off is quantified: LogStamping requires approximately 35% more verification time than BCALS at 10,000 entries, in exchange for 38% less storage — a favourable trade for storage-constrained deployments.

Five formally specified algorithms — governing log ingestion, SHA-256 hashing, blockchain submission, smart contract storage, and timestamp-aware integrity verification — provide a complete and reproducible specification of the system's operational logic. The semantic distinction between wall-clock ingest timeout (T_ing) and timestamp-derived verification timeout (T_ver) is identified as a critical design element that enables correct offline verification — a novel clarification not present in prior LogStamping literature.

The name LogStamping aptly captures the design philosophy: just as timestamp authorities in PKI systems anchor document integrity at a point in time through digital signatures, LogStamping anchors log group integrity at the moment of ingestion through blockchain-committed cryptographic digests. The result is a scalable, storage-

efficient, and forensically sound foundation for log integrity assurance in large-scale distributed systems.

4.2 Limitations

Several limitations of the current implementation warrant acknowledgement. First, the inability to recover log data during the ingestion window represents a significant operational risk: since raw log content is not retained until verification and IPFS archival are complete, any loss or corruption of the log file between generation and archival results in irreversible data loss. This design choice intentionally trades recoverability for privacy and on-chain storage minimization, but may not be acceptable in all operational contexts.

Second, the PoA trust model provides weaker adversarial guarantees than a public blockchain in multi-organizational deployment scenarios. Third, the timestamp parsing coverage of Algorithm 5 is limited to common log formats; entries with non-standard timestamps fall back to line-count-only grouping, potentially misaligning groups between ingest and verify phases. Fourth, the benchmark comparison relies on normalized values from published literature rather than direct re-implementation, which introduces uncertainty in the absolute cross-system comparison, though relative trends are well-supported.

4.3 Recommendations for Future Work

Based on the findings and limitations identified above, the following directions are recommended for future development:

10. Fault-tolerant ingestion pipeline: Implement write-ahead buffering (WAL) for ingestion agents to address the recoverability gap, ensuring that log groups are durably buffered before on-chain anchoring and IPFS archival are confirmed.
11. Formal smart contract verification: Apply formal verification tools (Certora Prover, Mythril, Slither) to the LogStorage contract to provide mathematical

guarantees of correctness and identify potential vulnerabilities before production deployment.

12. Anomaly detection integration: Integrate ML-based anomaly detection algorithms over Elasticsearch-indexed logs, enabling not only tamper detection (integrity) but also behavioural anomaly detection (security monitoring) within a unified platform.
13. Public chain anchoring: Implement periodic checkpointing of the Ethereum PoA state to a public chain (Ethereum mainnet or a Layer 2 rollup) to provide stronger adversarial trust guarantees without incurring full public chain transaction costs for every log group.
14. Extended timestamp parsing: Expand the timestamp parsing library to cover a broader range of log formats, or adopt a canonical log format (e.g., RFC 5424 syslog) for new deployments to eliminate the group misalignment risk.
15. Standardized benchmarking protocol: Develop and propose a community benchmarking protocol for blockchain log integrity systems, enabling fair direct comparison across implementations — addressing the cross-system comparison uncertainty identified in this work.
16. Multi-tenancy and identity attribution: Extend the smart contract to support decentralized identity-based audit attribution, enabling multi-tenant environments where log groups from multiple sources are anchored to a shared chain with provably distinct authorship.

REFERENCE LIST

[1] Md Shariful Islam and M. Sohel Rahman, "LogStamping: A Blockchain-Based Log Auditing Approach for Large-Scale Systems," arXiv preprint arXiv:2505.17236, 2025.

[2] A. Ali, A. Khan, M. Ahmed, and G. Jeon, "BCALS: Blockchain-Based Secure Log Management System for Cloud Computing," Transactions on Emerging Telecommunications Technologies, e4272, 2021.

[3] L. M. Shekhtman and E. Waisbard, "EngraveChain: A Blockchain-Based Tamper-Proof Distributed Log System," Future Internet, vol. 13, no. 6, p. 143, 2021.

[4] X. Liang, S. Shetty, D. Tosh, C. Kamhoua, K. Kwiat, and L. Njilla, "ProvChain: A Blockchain-Based Data Provenance Architecture in Cloud Environment with Enhanced Privacy and Availability," in Proc. 17th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Comput. (CCGrid), 2017, pp. 468–477.

[5] I. Ray, K. Belyaev, M. Strizhov, D. Mulamba, and M. Rajaram, "Secure Logging as a Service — Delegating Log Management to the Cloud," IEEE Systems Journal, vol. 7, no. 2, pp. 323–334, 2013.

[6] W. Pourmajidi and A. Miranskyy, "Logchain: Blockchain-Assisted Log Storage," in Proc. 11th IEEE Int. Conf. Cloud Computing (CLOUD), 2018, pp. 978–982.

[10] B. Schneier and J. Kelsey, "Cryptographic Support for Secure Logs on Untrusted Machines," in Proc. 7th USENIX Security Symposium, 1998.

[11] J. E. Holt, "Logcrypt: Forward Security and Public Verification for Secure Audit Logs," in Proc. AISW-NetSec, 2006.

[12] A. Ahmad, M. Saad, M. Bassiouni, and A. Mohaisen, "Towards Blockchain-Driven, Secure and Transparent Audit Logs," in Proc. MobiQuitous, 2018.

[14] IBM Corporation, "Storage Needs for Blockchain Technology: Point of View," IBM, 2018.

[19] M. H. Rakib, S. Hossain, M. Jahan, and U. Kabir, "A Blockchain-Enabled Scalable Network Log Management System," Journal of Computer Science, vol. 18, no. 6, pp. 496–508, 2022.

[25] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[26] V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum Whitepaper, 2013. [Online]. Available: https://ethereum.org/en/whitepaper/

[28] LogPai, "A Large Collection of System Log Datasets for AI-Driven Log Analytics," GitHub, 2024. [Online]. Available: https://github.com/logpai/loghub

[29] K. Basu, "Fake-Apache-Log-Generator," GitHub, 2024. [Online]. Available: https://github.com/kiritbasu/Fake-Apache-Log-Generator

[30] H. Yoshida and A. Biryukov, "Analysis of a SHA-256 Variant," in Advances in Cryptology — EUROCRYPT 2005, Springer, 2005, pp. 245–260.

[32] A. Chuvakin, K. Schmidt, and C. Phillips, Logging and Log Management: The Authoritative Guide. Syngress, 2013.

[33] K. Kent and M. Souppaya, "Guide to Computer Security Log Management," NIST Special Publication 800-92, 2006.

[34] A. Barisani and D. Oldani, Offensive Computing: Understanding Windows, Linux, and UNIX Security. Syngress, 2007.

[35] D. Yaga, P. Mell, N. Roby, and K. Scarfone, "Blockchain Technology Overview," NIST Internal Report 8202, 2018.

[37] Aptive, "Log Injection Attack: Understanding and Mitigating the Risks," 2024. [Online]. Available: https://www.aptive.co.uk/blog/log-injection-attack/

[41] G. Xu, F. Yun, S. Xu, Y. Yu, X.-B. Chen, and M. Dong, "A Blockchain-Based Log Storage Model with Efficient Query," Soft Computing, vol. 27, pp. 13779–13787, 2023.

[45] A. I. Sanka and R. C. C. Cheung, "A Systematic Review of Blockchain Scalability: Issues, Solutions, Analysis and Future Research," Journal of Network and Computer Applications, vol. 195, p. 103232, 2021.

[47] U.S. Department of Health & Human Services, "Health Insurance Portability and Accountability Act (HIPAA)," 1996. [Online]. Available: https://www.hhs.gov/hipaa/index.html

[48] European Union, "General Data Protection Regulation (GDPR)," Regulation (EU) 2016/679, 2016. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[49] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[50] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.180-4

[51] Ethereum Foundation, "Solidity: High-Level Language for Smart Contracts," 2024. [Online]. Available: https://soliditylang.org

[52] V. Buterin, "Proof of Authority Chains," Ethereum Foundation, 2017. [Online]. Available: https://github.com/ethereum/EIPs/issues/225

APPENDICES

Appendix A: LogStorage Smart Contract Source Code (Solidity)

The following is the complete Solidity source code for the LogStorage smart contract deployed on the private Ethereum PoA network:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract LogStorage {
    mapping(uint256 => string) private logHashes;
    uint256 public logCount;
    address private owner;
    constructor() { owner = msg.sender; }
    modifier onlyOwner() { require(msg.sender == owner); _; }
    function storeLogHash(string memory h) public onlyOwner {
        logHashes[logCount] = h; logCount++;
    }
    function getLogHash(uint256 i) public view returns (string
memory) {
        return logHashes[i];
    }
    function hasLogHash(string memory h) public view returns (bool)
{
        for (uint256 i = 0; i < logCount; i++) {
            if (keccak256(bytes(logHashes[i])) ==
keccak256(bytes(h))) return true;
        } return false;
    }
}
```

Appendix B: Docker Compose Configuration

The complete Docker Compose YAML configuration for the three-node Ethereum PoA network, Elasticsearch, IPFS daemon, and FastAPI management interface is available in the project repository. The configuration file specifies node volumes (200 GB each), networking, environment variables for Geth initialization, and health check endpoints for each service. Each Geth node is initialized from a shared genesis.json file specifying the Clique consensus parameters and the initial authorized signer addresses.

Appendix C: Sample Log Dataset and Benchmark Script

Benchmark scripts were implemented in Python using the time and subprocess libraries. Each script executes a complete ingestion or verification run, records wall-clock timing using Python's time.perf_counter() for high-resolution measurement, and captures Geth datadir size before and after ingestion using os.path.getsize() on the data directory. Sample log entries from the LogPai benchmark collection [28] and synthetically generated Apache access log entries from Fake-Apache-Log-Generator [29] were used as the evaluation datasets. The synthetic generator was configured to produce entries with realistic IP distributions, timestamp sequences, and HTTP status code proportions representative of production web server traffic.